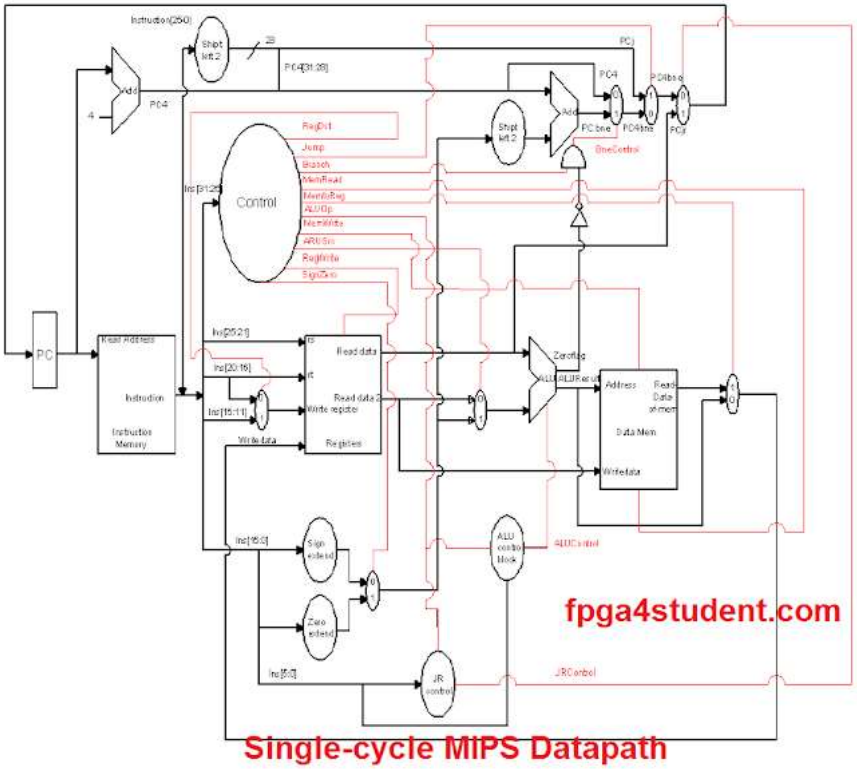
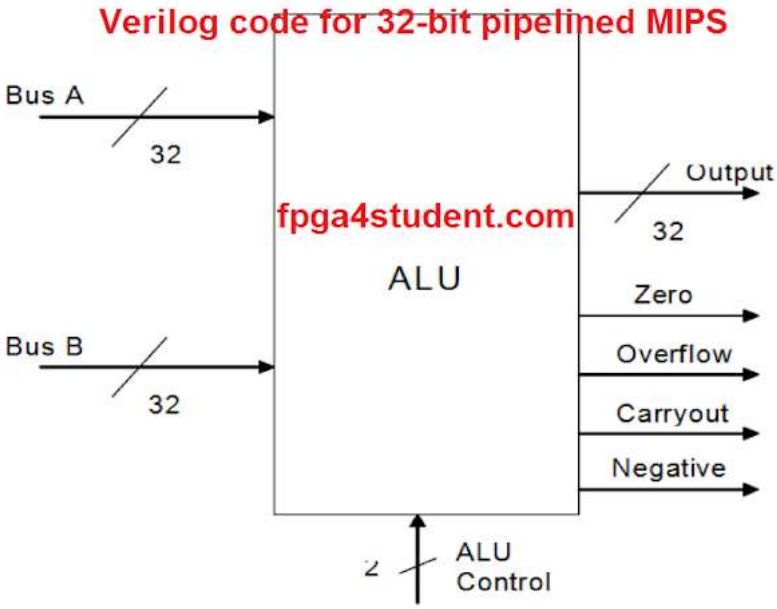
Pipelined MIPS Processor in Verilog (Part-2)

This project is to present the Verilog code for 32-bit 5-stage pipelined MIPS Processor.
In part 1, I presented the instruction set of the pipelined MIPS processor and partially provided the Verilog code for the single-cycle MIPS datapath as shown below. Now, continue on the design and Verilog code for the ALU of the MIPS processor.



ALU Block Interface of the MIPS Processor:



Verilog code for 32-bit ALU:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
```

```
// Verilog code for ALU
module alu(Output, CarryOut, zero, overflow, negative, BussA, BussB, ALUControl);
output CarryOut,overflow,negative,zero;
output [31:0] Output;
input [31:0] BussA,BussB;
input [1:0] ALUControl;
wire lessthan;
wire [31:0] crrout;

alu1bit alu0(Output[0],crrout[0],BussA[0],BussB[0],ALUControl[1],lessthan,ALUContr
alu1bit alu1(Output[1],crrout[1],BussA[1],BussB[1],crrout[0],1'b0,ALUControl);
alu1bit alu2(Output[2],crrout[2],BussA[2],BussB[2],crrout[1],1'b0,ALUControl);
alu1bit alu3(Output[3],crrout[3],BussA[3],BussB[3],crrout[2],1'b0,ALUControl);
alu1bit alu4(Output[4],crrout[4],BussA[4],BussB[4],crrout[3],1'b0,ALUControl);
alu1bit alu5(Output[5],crrout[5],BussA[5],BussB[5],crrout[4],1'b0,ALUControl);
alu1bit alu6(Output[6],crrout[6],BussA[6],BussB[6],crrout[5],1'b0,ALUControl);
alu1bit alu7(Output[7],crrout[7],BussA[7],BussB[7],crrout[6],1'b0,ALUControl);
alu1bit alu8(Output[8],crrout[8],BussA[8],BussB[8],crrout[7],1'b0,ALUControl);
alu1bit alu9(Output[9],crrout[9],BussA[9],BussB[9],crrout[8],1'b0,ALUControl);
alu1bit alu10(Output[10],crrout[10],BussA[10],BussB[10],crrout[9],1'b0,ALUControl)
alu1bit alu11(Output[11],crrout[11],BussA[11],BussB[11],crrout[10],1'b0,ALUControl
alu1bit alu12(Output[12],crrout[12],BussA[12],BussB[12],crrout[11],1'b0,ALUControl
alu1bit alu13(Output[13],crrout[13],BussA[13],BussB[13],crrout[12],1'b0,ALUControl
alu1bit alu14(Output[14],crrout[14],BussA[14],BussB[14],crrout[13],1'b0,ALUControl
alu1bit alu15(Output[15],crrout[15],BussA[15],BussB[15],crrout[14],1'b0,ALUControl
alu1bit alu16(Output[16],crrout[16],BussA[16],BussB[16],crrout[15],1'b0,ALUControl
alu1bit alu17(Output[17],crrout[17],BussA[17],BussB[17],crrout[16],1'b0,ALUControl
alu1bit alu18(Output[18],crrout[18],BussA[18],BussB[18],crrout[17],1'b0,ALUControl
alu1bit alu19(Output[19],crrout[19],BussA[19],BussB[19],crrout[18],1'b0,ALUControl
alu1bit alu20(Output[20],crrout[20],BussA[20],BussB[20],crrout[19],1'b0,ALUControl
alu1bit alu21(Output[21],crrout[21],BussA[21],BussB[21],crrout[20],1'b0,ALUControl
alu1bit alu22(Output[22],crrout[22],BussA[22],BussB[22],crrout[21],1'b0,ALUControl
alu1bit alu23(Output[23],crrout[23],BussA[23],BussB[23],crrout[22],1'b0,ALUControl
alu1bit alu24(Output[24],crrout[24],BussA[24],BussB[24],crrout[23],1'b0,ALUControl
alu1bit alu25(Output[25],crrout[25],BussA[25],BussB[25],crrout[24],1'b0,ALUControl
alu1bit alu26(Output[26],crrout[26],BussA[26],BussB[26],crrout[25],1'b0,ALUControl
alu1bit alu27(Output[27],crrout[27],BussA[27],BussB[26],crrout[26],1'b0,ALUControl
alu1bit alu28(Output[28],crrout[28],BussA[28],BussB[28],crrout[27],1'b0,ALUControl
alu1bit alu29(Output[29],crrout[29],BussA[29],BussB[29],crrout[28],1'b0,ALUControl
alu1bit alu30(Output[30],crrout[30],BussA[30],BussB[30],crrout[29],1'b0,ALUControl
alu1bit alu31(Output[31],crrout[31],BussA[31],BussB[31],crrout[30],1'b0,ALUControl
not #(50) notcarry(notcr31,crrout[31]);
// Carryout = Not carry out 31 if it is subtraction
mux21 muxcarry31(CarryOut,crrout[31],notcr31,ALUControl[1]);
xor #(50) xor5(overflow,crrout[30],crrout[31]);
// SLT
addsub add2(addsub31Out,crrout31,BussA[31],BussB[31],crrout[30],ALUControl[1]);
xor #(50) xor6(lessthan,overflow,addsub31Out);
assign negative = Output[31];
or #(50) or1(o1,Output[0],Output[1],Output[2],Output[3]);
or #(50) or2(o2,Output[4],Output[5],Output[6],Output[7]);
or #(50) or3(o3,Output[8],Output[9],Output[10],Output[11]);
or #(50) or4(o4,Output[12],Output[13],Output[14],Output[15]);
or #(50) or5(o5,Output[16],Output[17],Output[18],Output[19]);
or #(50) or6(o6,Output[20],Output[21],Output[22],Output[23]);
or #(50) or7(o7,Output[24],Output[25],Output[26],Output[27]);
or #(50) or8(o8,Output[28],Output[29],Output[30],Output[31]);
or #(50) or9(o9,o1,o2,o3,o4);
or #(50) or10(o10,o5,o6,o7,o8);
nor #(50) nor1(zero,o9,o10);
endmodule
`timescale 1 ps / 100 fs
module alu1bit(result,crrout,a,b,carryin,less,ALUControl);
```

```
output result,crrout;
input a,b,carryin,less;
input [1:0] ALUControl;
addsub add1(addsubOut,crrout,a,b,carryin,ALUControl[1]);
xor #(50) xor1(xorOut,a,b);
mux21 mux2(xorlessOut,xorOut,less,ALUControl[1]);
mux21 mux3(result,addsubOut,xorlessOut,ALUControl[0]);
endmodule

`timescale 1 ps / 100 fs
module addsub(Out,cout,a,b,cin,select);
input a,b,cin,select;
output Out,cout; // the result and carry out
not #(50) not1(notb,b);
mux21 mux1(b1,b,notb,select);

adder adder1(Out,cout,a,b1,cin);
endmodule

`timescale 1 ps / 100 fs
module adder(sum,cout,a,b,cin);
input   a,b,cin;
output  cout,sum;
// sum = a xor b xor cin
xor #(50) (sum,a,b,cin);
// carry out = a.b + cin.(a+b)
and #(50) and1(c1,a,b);
or #(50) or1(c2,a,b);
and #(50) and2(c3,c2,cin);
or #(50) or2(cout,c1,c3);
endmodule

`timescale 1 ps / 100 fs
module mux21(O,A,B,sel);
// sel = 0 thi O = A
// sel = 1 thi O =B
output O;
input A,B,sel;
not #(50) not1(nsel,sel);
and #(50) and1(O1,A,nsel);
and #(50) and2(O2,B,sel);
or #(50) or2(O,O1,O2);
endmodule
```

Verilog code for Data Memory:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// Data Memory
module dataMem(data, address, writedata, writeenable,MemRead,clk);

input [31:0] address, writedata;
input writeenable,MemRead, clk;
output [31:0] data;
reg [7:0] datamem[1023:0];
reg [31:0] temp;

buf #1000 buf0(data[0],temp[0]),
   buf1(data[1],temp[1]),
   buf2(data[2],temp[2]),
   buf3(data[3],temp[3]),
   buf4(data[4],temp[4]),
   buf5(data[5],temp[5]),
```

```
   buf6(data[6],temp[6]),
   buf7(data[7],temp[7]),
   buf8(data[8],temp[8]),
   buf9(data[9],temp[9]),
   buf10(data[10],temp[10]),
   buf11(data[11],temp[11]),
   buf12(data[12],temp[12]),
   buf13(data[13],temp[13]),
   buf14(data[14],temp[14]),
   buf15(data[15],temp[15]),
   buf16(data[16],temp[16]),
   buf17(data[17],temp[17]),
   buf18(data[18],temp[18]),
   buf19(data[19],temp[19]),
   buf20(data[20],temp[20]),
   buf21(data[21],temp[21]),
   buf22(data[22],temp[22]),
   buf23(data[23],temp[23]),
   buf24(data[24],temp[24]),
   buf25(data[25],temp[25]),
   buf26(data[26],temp[26]),
   buf27(data[27],temp[27]),
   buf28(data[28],temp[28]),
   buf29(data[29],temp[29]),
   buf30(data[30],temp[30]),
   buf31(data[31],temp[31]);

always @(posedge clk)
 if (writeenable)
 begin
  datamem[address]=writedata[31:24];
  datamem[address+1]=writedata[23:16];
  datamem[address+2]=writedata[15:8];
  datamem[address+3]=writedata[7:0];
 end

always @(address or datamem[address] or datamem[address+1] or datamem[address+2] o
begin
 temp={datamem[address],datamem[address+1],datamem[address+2],datamem[address+3]};
end

// initial
// begin
// $readmemh("data.dat", datamem);
// end

endmodule
```

Verilog code for Zero-Extension module:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// Zero-Extension
module zero_extend(zOut32,zIn16);
output [31:0] zOut32;
input [15:0] zIn16;
assign zOut32 = {{16{1'b0}},zIn16};
endmodule
```

Verilog code of the Multiplexer for choosing Bus B ALU:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
```

```
// mux2x32to32
module mux2x32to32( DataOut,Data0, Data1, Select);
output [31:0] DataOut; // Data Out
input [31:0] Data0, Data1; // Data In 1 and 2
input Select;
// neu Select = 0 thi DataOut = Data0
// nguoc lai thi DataOut = Data1

mux2_1 mux0(DataOut[0],Data0[0],Data1[0],Select);
mux2_1 mux1(DataOut[1],Data0[1],Data1[1],Select);
mux2_1 mux2(DataOut[2],Data0[2],Data1[2],Select);
mux2_1 mux3(DataOut[3],Data0[3],Data1[3],Select);
mux2_1 mux4(DataOut[4],Data0[4],Data1[4],Select);
mux2_1 mux5(DataOut[5],Data0[5],Data1[5],Select);
mux2_1 mux6(DataOut[6],Data0[6],Data1[6],Select);
mux2_1 mux7(DataOut[7],Data0[7],Data1[7],Select);
mux2_1 mux8(DataOut[8],Data0[8],Data1[8],Select);
mux2_1 mux9(DataOut[9],Data0[9],Data1[9],Select);
mux2_1 mux10(DataOut[10],Data0[10],Data1[10],Select);
mux2_1 mux11(DataOut[11],Data0[11],Data1[11],Select);
mux2_1 mux12(DataOut[12],Data0[12],Data1[12],Select);
mux2_1 mux13(DataOut[13],Data0[13],Data1[13],Select);
mux2_1 mux14(DataOut[14],Data0[14],Data1[14],Select);
mux2_1 mux15(DataOut[15],Data0[15],Data1[15],Select);
mux2_1 mux16(DataOut[16],Data0[16],Data1[16],Select);
mux2_1 mux17(DataOut[17],Data0[17],Data1[17],Select);
mux2_1 mux18(DataOut[18],Data0[18],Data1[18],Select);
mux2_1 mux19(DataOut[19],Data0[19],Data1[19],Select);
mux2_1 mux20(DataOut[20],Data0[20],Data1[20],Select);
mux2_1 mux21(DataOut[21],Data0[21],Data1[21],Select);
mux2_1 mux22(DataOut[22],Data0[22],Data1[22],Select);
mux2_1 mux23(DataOut[23],Data0[23],Data1[23],Select);
mux2_1 mux24(DataOut[24],Data0[24],Data1[24],Select);
mux2_1 mux25(DataOut[25],Data0[25],Data1[25],Select);
mux2_1 mux26(DataOut[26],Data0[26],Data1[26],Select);
mux2_1 mux27(DataOut[27],Data0[27],Data1[27],Select);
mux2_1 mux28(DataOut[28],Data0[28],Data1[28],Select);
mux2_1 mux29(DataOut[29],Data0[29],Data1[29],Select);
mux2_1 mux30(DataOut[30],Data0[30],Data1[30],Select);
mux2_1 mux31(DataOut[31],Data0[31],Data1[31],Select);

endmodule
```

Verilog code for Sign Extension and Shift Left 2 Module:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// Sign-Extension
module sign_extend(sOut32,sIn16);
output [31:0] sOut32;
input [15:0] sIn16;
assign sOut32 = {{16{sIn16[15]}},sIn16};
endmodule
// Shift left 2 module
module shift_left_2(Out32, In32);
output [31:0] Out32;
input [31:0] In32;

assign Out32 = {In32[29:0],2'b00};
endmodule
```

Verilog code for the Multiplexer choosing write address:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
```

```
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// mux2x5to5
// Select Write Register
module mux2x5to5( AddrOut,Addr0, Addr1, Select);
output [4:0] AddrOut; // Address Out
input [4:0] Addr0, Addr1; // Address In 1 and 2
input Select;
mux21 mux0(AddrOut[0],Addr0[0],Addr1[0],Select);
mux21 mux1(AddrOut[1],Addr0[1],Addr1[1],Select);
mux21 mux2(AddrOut[2],Addr0[2],Addr1[2],Select);
mux21 mux3(AddrOut[3],Addr0[3],Addr1[3],Select);
mux21 mux4(AddrOut[4],Addr0[4],Addr1[4],Select);
endmodule
module mux2_1(O,A,B,sel);
// sel = 0 : O = A
// sel = 1 : O =B
output O;
input A,B,sel;
not #(50) not1(nsel,sel);
and #(50) and1(O1,A,nsel);
and #(50) and2(O2,B,sel);
or #(50) or2(O,O1,O2);
endmodule
```

The Control signals of the Control unit are designed in the following table:

	Control Signals									
Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	MemRead	Memwrite	Branch	ALUOp1	ALUOp0	SignZero
R-type	1	0	0	1	0	0	0	1	0	0
LW	0	1	1	1	1	0	0	0	0	0
SW	0	1	0	0	0	1	0	0	0	0
BNE	0	0	0	0	0	0	1	0	1	0
XORI	0	1	0	1	0	0	0	1	1	1
J	0	0	0	0	0	0	0	0	0	0

fpga4student.com

Verilog code for the Control Unit:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// Control unit
module Control(
RegDst,
ALUSrc,
MemtoReg,
RegWrite,
MemRead,
MemWrite,
Branch,
ALUOp,
Jump,
SignZero,
Opcode
);

output RegDst,ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,SignZero;
output [1:0] ALUOp;
input [5:0] Opcode;
reg RegDst,ALUSrc,MemtoReg,RegWrite,MemRead,MemWrite,Branch,Jump,SignZero;
reg [1:0] ALUOp;
always @(*)
casex (Opcode)
 6'b000000 : begin // R - type
```

```
    RegDst = 1'b1;
    ALUSrc = 1'b0;
    MemtoReg= 1'b0;
    RegWrite= 1'b1;
    MemRead = 1'b0;
    MemWrite= 1'b0;
    Branch = 1'b0;
    ALUOp = 2'b10;
    Jump = 1'b0;
    SignZero= 1'b0;
   end
6'b100011 : begin // lw - load word
    RegDst = 1'b0;
    ALUSrc = 1'b1;
    MemtoReg= 1'b1;
    RegWrite= 1'b1;
    MemRead = 1'b1;
    MemWrite= 1'b0;
    Branch = 1'b0;
    ALUOp = 2'b00;
    Jump = 1'b0;
    SignZero= 1'b0; // sign extend
   end
6'b101011 : begin // sw - store word
    RegDst = 1'bx;
    ALUSrc = 1'b1;
    MemtoReg= 1'bx;
    RegWrite= 1'b0;
    MemRead = 1'b0;
    MemWrite= 1'b1;
    Branch = 1'b0;
    ALUOp = 2'b00;
    Jump = 1'b0;
    SignZero= 1'b0;
   end
6'b000101 : begin // bne - branch if not equal
    RegDst = 1'b0;
    ALUSrc = 1'b0;
    MemtoReg= 1'b0;
    RegWrite= 1'b0;
    MemRead = 1'b0;
    MemWrite= 1'b0;
    Branch = 1'b1;
    ALUOp = 2'b01;
    Jump = 1'b0;
    SignZero= 1'b0; // sign extend
   end
6'b001110 : begin // XORI - XOR immidiate
    RegDst = 1'b0;
    ALUSrc = 1'b1;
    MemtoReg= 1'b0;
    RegWrite= 1'b1;
    MemRead = 1'b0;
    MemWrite= 1'b0;
    Branch = 1'b0;
    ALUOp = 2'b11;
    Jump = 1'b0;
    SignZero= 1'b1; // zero extend
   end
6'b000010 : begin // j - Jump
    RegDst = 1'b0;
    ALUSrc = 1'b0;
    MemtoReg= 1'b0;
```

```
      RegWrite= 1'b0;
      MemRead = 1'b0;
      MemWrite= 1'b0;
      Branch = 1'b0;
      ALUOp = 2'b00;
      Jump = 1'b1;
      SignZero= 1'b0;
     end
 default : begin
      RegDst = 1'b0;
      ALUSrc = 1'b0;
      MemtoReg= 1'b0;
      RegWrite= 1'b0;
      MemRead = 1'b0;
      MemWrite= 1'b0;
      Branch = 1'b0;
      ALUOp = 2'b10;
      Jump = 1'b0;
      SignZero= 1'b0;
     end

endcase

endmodule
```

ALU Control Signals are designed as follows:

ALU Control Signals				
ALUOp	Function	ALUControl	ALU Operation	Instruction
11	xxxxxx	01	XOR	XORI
00	xxxxxx	00	Add	LW - SW
01	xxxxxx	10	Sub	BNE
10	100000	00	Add	R-type: ADD
10	100010	10	Sub	R-type: SUB
10	101010	11	SLT	R-type: SLT

fpga4student.com

Verilog code for ALU Control Unit:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// ALU Control unit
module ALUControl_Block( ALUControl, ALUOp, Function);
output [1:0] ALUControl;
reg [1:0] ALUControl;
input [1:0] ALUOp;
input [5:0] Function;
wire [7:0] ALUControlIn;
assign ALUControlIn = {ALUOp,Function};
always @(ALUControlIn)
casex (ALUControlIn)
 8'b11xxxxxx: ALUControl=2'b01;
 8'b00xxxxxx: ALUControl=2'b00;
 8'b01xxxxxx: ALUControl=2'b10;
 8'b10100000: ALUControl=2'b00;
 8'b10100010: ALUControl=2'b10;
 8'b10101010: ALUControl=2'b11;
 default: ALUControl=2'b00;
 endcase
endmodule
```

Verilog code for JR Control Unit:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// Control singals for JR instruction
module JRControl_Block( JRControl, ALUOp, Function);
output JRControl;
reg JRControl;
input [1:0] ALUOp;
input [5:0] Function;
wire [7:0] test;
assign test = {ALUOp,Function};
always @(test)
case (test)
 8'b10001000 : JRControl=1'b1;
 default: JRControl=1'b0;
 endcase
endmodule
```

Basically, we have done the Verilog code for the single-cycle MIPS Processor. In the next part, pipelined registers are added to the single-cycle datapath and make it become a pipelined MIPS Processor. Special designs such as forwarding, stall control, and flush control unit are needed to solve hazards in the pipelined MIPS Processor.

Continue on Part 3...

It is noted that you need to go through all the necessary parts(Part 1, Part 2, and Part 3) to fully understand the process of designing the pipelined MIPS processor, and collect all the required Verilog code to be able to run the pipelined MIPS processor in simulation.

You may like this:

Verilog code for a Microcontroller
A complete 8-bit Microcontroller in VHDL
Single-cycle MIPS Processor in Verilog
Verilog Code for 16-bit RISC Processor
VHDL code for MIPS processor

What is FPGA Programming? FPGA vs Software programming
Recommended and affordable Xilinx FPGA boards for students
Recommended and affordable Altera FPGA boards for students

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor

22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. How to generate a clock enable signal in Verilog
34. Verilog code for PWM Generator
35. Verilog coding vs Software Programming
36. Verilog code for Sequence Detector using Moore FSM

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.